

Week 4 - Creating and Managing ROS2 Workspaces and Packages

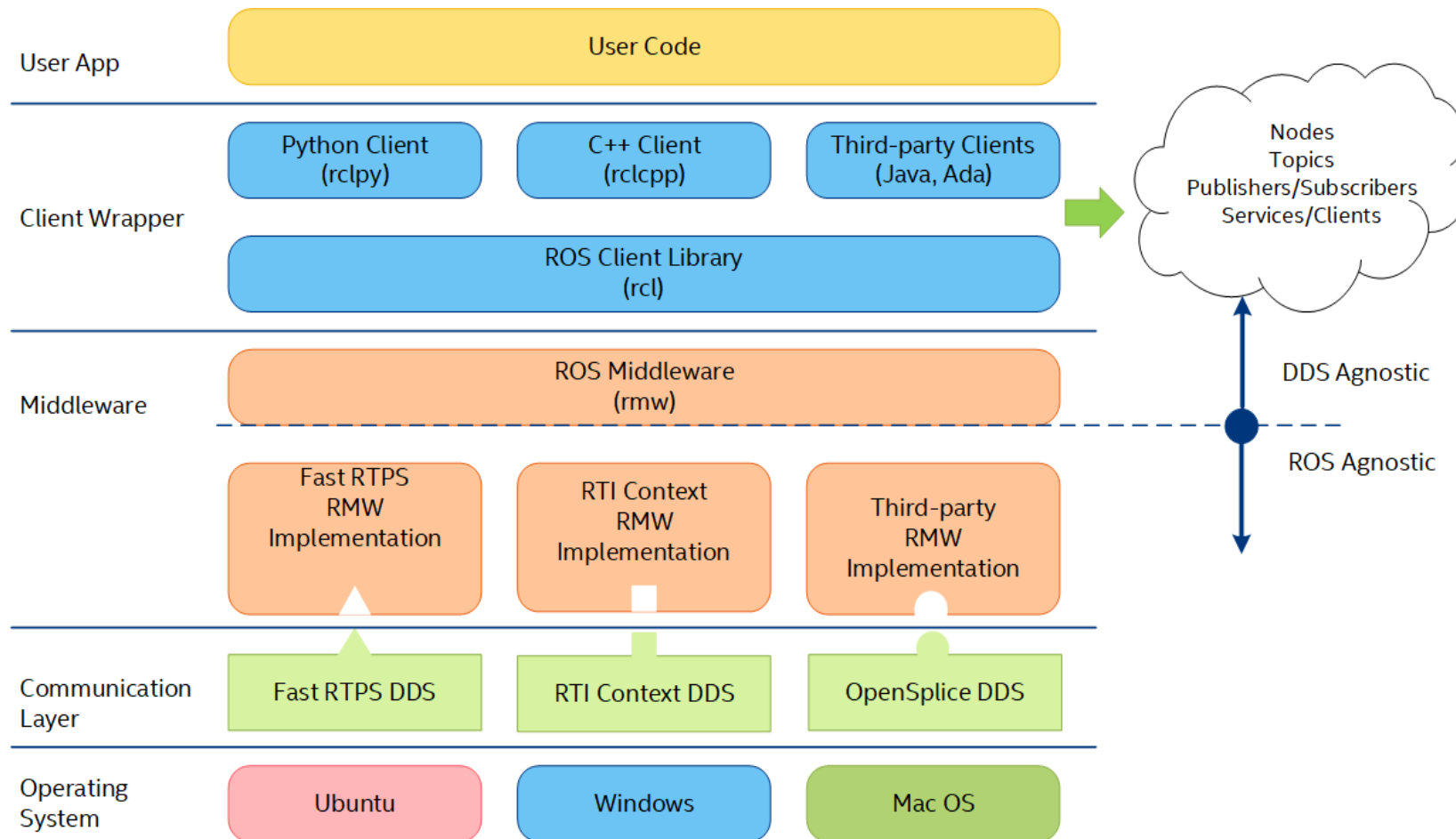
ROBOTIC EMBEDDED PROGRAMMING
MENG 4060

ANDY ALUBAIDY, PHD, P.ENG.

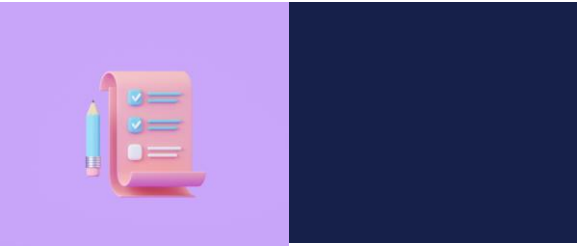


**WE ARE
HUMBER**

Review from Last Week



Review from Last Week



User APP

```
class Talker(Node):
    def __init__(self):
        super().__init__('talker')
        self.i = 0
        timer_period = 1.0
        self.tmr = self.create_timer(timer_period, self.timer_callback)
    def timer_callback(self):
        msg = String()
        msg.data = 'Hello World: {0}'.format(self.i)
        self.i += 1
        self.get_logger().info('Publishing: "{0}"'.format(msg.data))

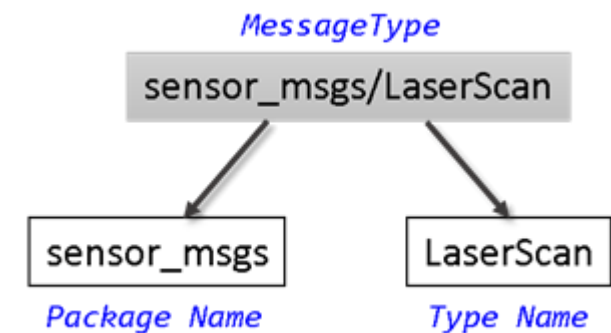
def main(args=None):
    node = Talker()
    try:
        rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
    finally:
        node.destroy_node()
        rclpy.try_shutdown()
```

Client Wrapper	import rclpy from rclpy.executors import ExternalShutdownException from rclpy.node import Node from std_msgs.msg import String
Middleware	self.pub = self.create_publisher(String, 'chatter', 10) self.pub.publish(msg) rclpy.init(args=args) rclpy.spin(node)
Communication Layer	Fast DDS
Operating System	Ubuntu

Review from Last Week



- **ROS *messages*** are the primary container for exchanging data in ROS 2.
- **Publishers and subscribers exchange data using messages on specified *topics* to carry data between nodes.**
- **To identify its data structure, each message has a message type.**
 - For example, sensor data from a laser scanner is typically sent in a message of type `sensor_msgs/LaserScan`.
 - Each message type identifies the data elements that are contained in a message.
 - Every message type name is a combination of a package name, followed by a forward slash /, and a type name:



<https://www.mathworks.com/help/ros/ug/work-with-basic-ros-2-messages.html>

Agenda for today's training session



Performance Objective:

- Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions
- Configure ROS2 on a Linux-based system
- Develop ROS2 nodes in Python or C++ using rclpy or rclcpp
- Manage ROS2 workspaces, packages and files by applying structured software development and modular design practices

Problem we will be solving today:

- Workspace
- Package

What is a ROS2 Workspace



- **A workspace is a directory structure where ROS2 packages are developed and built.**
 - **Enables modular development and separation of concerns.**
 - **Helps manage multiple packages in a single location.**
 - **Includes all the tools and processes that allow us to launch an application**

What is a ROS2 Workspace



- A workspace is where we build and organize our ROS 2 code.
- Contains:
 - **src/** → Source code (packages)
 - **build/** → Build system files
 - **install/** → Installed executables & libraries
- We can have multiple workspaces (e.g., dev_ws, robot_ws).

What is a ROS2 Workspace



- **Standard layout:**

```
ros2_ws/  
├── src/           # Source packages go here  
├── build/         # Temporary build files  
├── install/       # Executable outputs, source this folder  
└── log/          # Build logs
```

- **Usually built in the home directory /User/Home/ros2_ws.**
- **This workspace has to be activated so that packages it contains are available to use.**

What is a ROS2 Workspace



- **Standard layout breakdown:**

- **src Folder**

- **Stores all ROS 2 packages (Python or C++)**

- **Each package contains:**

- **Nodes (your ROS programs)**
 - **Launch files (to start multiple nodes)**
 - **Config files (parameters, settings)**

```
ros2_ws/  
├─ src/      # Source packages go here  
├─ build/    # Temporary build files  
├─ install/  # Executable outputs, source this folder  
└─ log/      # Build logs
```

What is a ROS2 Workspace



- **Standard layout breakdown:**

- **build Folder**

- **Stores intermediate compilation files (Temporary files) and metadata.**
 - **Includes test binaries and CMake artifacts.**

```
ros2_ws/  
├─ src/           # Source packages go here  
├─ build/         # Temporary build files  
├─ install/       # Executable outputs, source this folder  
└─ log/           # Build logs
```

What is a ROS2 Workspace



○install Folder

- Contains final executables and runtime files.
- Used for sourcing the environment (setup.bash)

○log Folder

- Records build and test output.
- Useful for debugging and tracking build steps.

```
ros2_ws/  
├─ src/      # Source packages go here  
├─ build/    # Temporary build files  
├─ install/  # Executable outputs, source this folder  
└─ log/      # Build logs
```

Workspace: Common Naming Convent



- Base station: **dev_ws**
- Robot (e.g., Raspberry Pi):
robot_ws
- Typically stored in **~/home**
directory

Workspace: Creating a Workspace



- Commands to run (example for dev_ws):

```
mkdir -p ~/dev_ws/src  
cd ~/dev_ws  
colcon build --symlink-install
```

1. `mkdir -p ~/dev_ws/src`
→ Make workspace & source folder
2. `cd ~/dev_ws`
→ Navigate into workspace
3. `colcon build --symlink-install`
→ Build workspace (even if empty)

Colcon



- **Colcon is the official ROS2 build tool (replaces catkin_make from ROS1).**
 - **Builds multiple packages in dependency order.**
 - **Supports build types:** ament_cmake, ament_python.
 - **Allows for selective rebuilding of packages using `--packages-select` flag.**

```
colcon build
```

```
colcon build --packages-select <package_name>
```

Colcon



- **What is Ament?**
 - **ament is the underlying build system in ROS2 defining how each package should be built.**
 - **Two types:**
 - **ament_cmake: for C++ packages**
 - **ament_python: for Python packages**
 - **Works with colcon to manage build logic, testing, and manage dependencies.**
 - **Defined in CMakeLists.txt or setup.py.**

Colcon - Installation

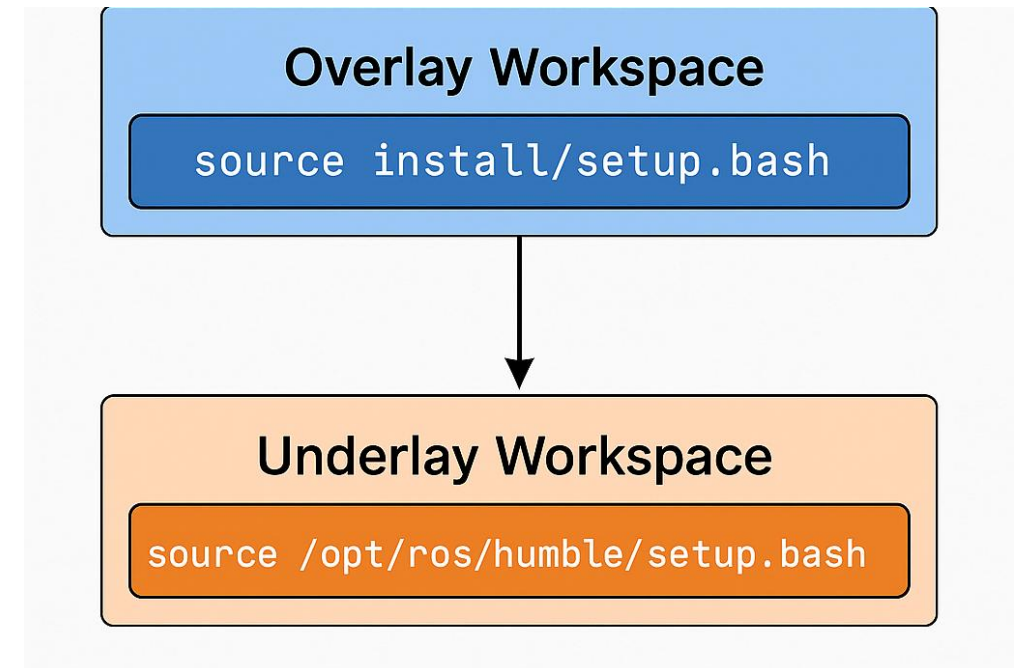


- **Before installing anything, always update your package list:**
`sudo apt update`
- **Install colcon extensions**
`sudo apt install python3-colcon-common-extensions`

ROS2 Underlay vs Overlay



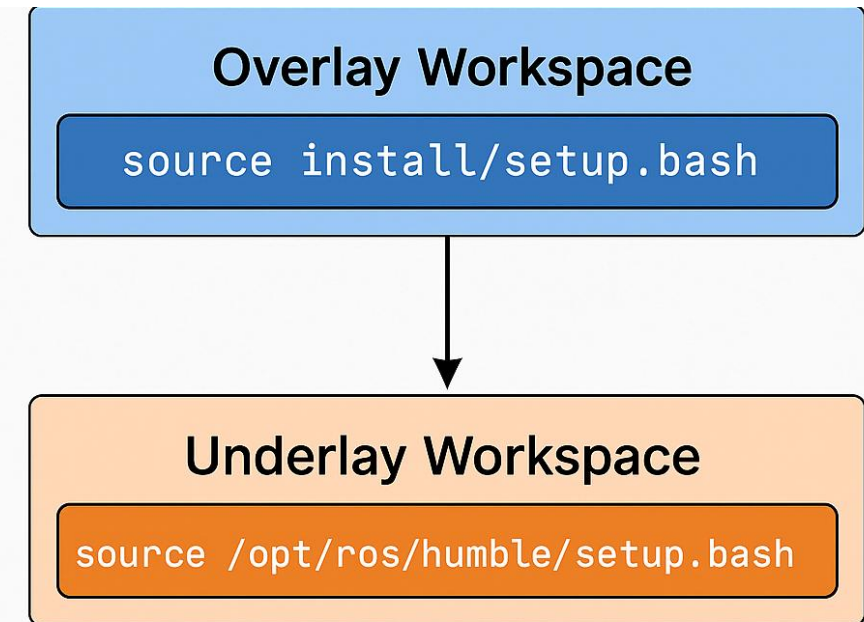
- **What is an Underlay Workspace?**
 - **The base environment that contains ROS2 core tools and libraries.**
 - **Typically, the installation from `/opt/ros/<distro-version>` is your underlay.**
 - **To activate the underlay:**
 - `source /opt/ros/<distro>/setup.bash`



ROS2 Underlay vs Overlay



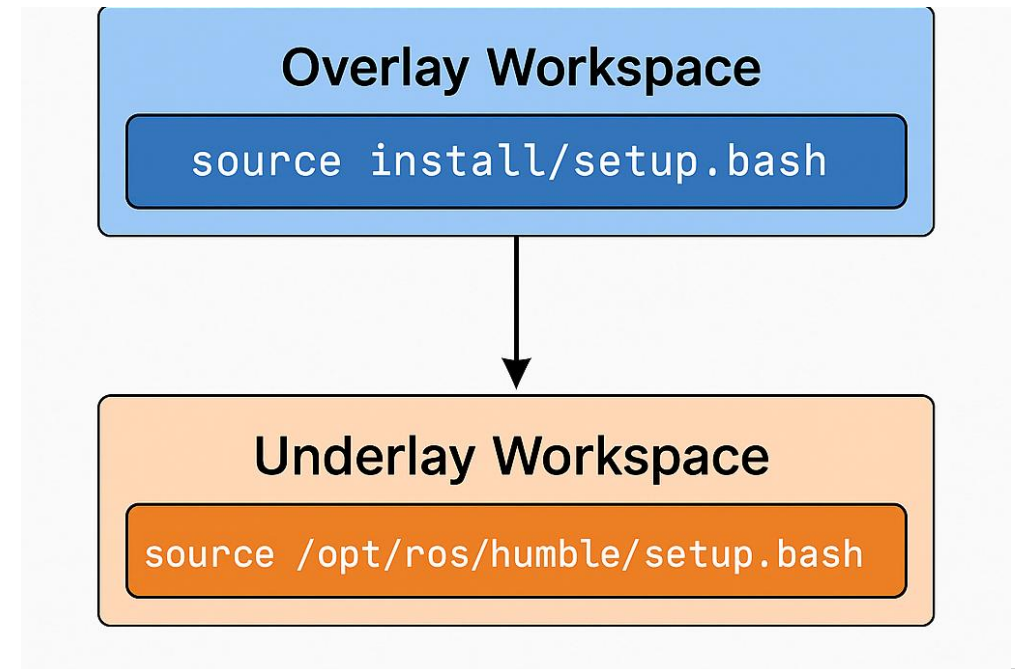
- **What is an Overlay Workspace?**
 - **A custom workspace layered on top of the underlay.**
 - **Contains your packages, nodes, and launch files.**
 - **You build and then source this workspace:**
 - `colcon build, in the workspace root directory (/User/Home/ros2_ws).`
 - `source ~/ros2_ws/install/setup.bash`



ROS2 Underlay vs Overlay



- **How do they work?**
 - **Overlay "adds" your packages to the environment without modifying the underlay.**
 - **You can have multiple overlay layers, if each is sourced in order.**
 - **Changes in your overlay don't affect the underlay installation — ideal for development!**
 - **Same packages in overlay as underlay will take precedence.**



How to create ROS2 Workspace



Colcon (I Do)



- **Create the workspace:**

- `mkdir -p ~/ros2_ws/src`
- `cd ~/ros2_ws`

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

- **Add a package to src:**

- `cd ~/ros2_ws/src`
- `git clone https://github.com/ros/ros\_tutorials.git -b jazzy`

```
cd ~/ros2_ws/src
```

```
# Clone the turtlesim source for ROS 2 Jazzy
git clone https://github.com/ros/ros_tutorials.git -b jazzy
```

- **Build the workspace:**

- `cd ~/ros2_ws`
- `colcon build --packages-select turtlesim`

```
# Build the package
cd ~/ros2_ws
colcon build --packages-select turtlesim
```

Colcon (I Do)



- Sometimes before building the workspace, you must check for dependencies that are missing:

- `rosdep install -i --from-path src --rosdistro jazzy -y`

```
$ rosdep install -i --from-path src --rosdistro jazzy -y
```

- Build the workspace:

- `cd ~/ros2_ws`
 - `colcon build --packages-select turtlesim`

```
# Build the package
```

```
cd ~/ros2_ws
```

```
colcon build --packages-select turtlesim
```

Colcon (You Do)



- **Follow the steps from the previous slides, then source your workspace and run the package.**
 - **source install/setup.bash**
 - **ros2 run turtlesim turtlesim_node**

```
# Source the workspace
```

```
source install/setup.bash
```

```
# Run turtlesim
```

```
ros2 run turtlesim turtlesim_node
```

What is a ROS2 Package



- **A package is the smallest buildable and runnable ROS2 unit.**
- **Contains:**
 - **Nodes**
 - **Configuration files**
 - **Launch files**
- **Must have:**
 - **package.xml: metadata and dependencies**
 - **setup.py or CMakeLists.txt: build instructions**



What is a ROS2 Package



- **A package can be created with two different build styles:**
 - `ament_cmake`, `ament_python`
 - `ament_cmake`: for C++ packages
 - `ament_python`: for Python packages

What is a ROS2 Package



- The following command can be used in Workspace/src folder to create a package:
 - **ros2 pkg create --build-type ament_python my_pkg --dependencies rclpy std_msgs**

```
ros2 pkg create --build-type ament_python my_pkg --dependencies rclpy std_msgs
```

- **--build-type**, defines a C++ or Python package structure
- **--dependencies**, are packages that your custom package relies on, to be installed in WS in the order defined

ROS2 Package Structure



```

ros2_ws/
└─ src/
    └─ my_robot_pkg/
        └─ my_robot_pkg/
            └─ __init__.py
            └─ my_node.py
        └─ package.xml
        └─ setup.py
        └─ setup.cfg
        └─ resource/
            └─ my_robot_pkg
        └─ launch/
            └─ my_launch.py
        └─ config/
            └─ params.yaml
        └─ test/
            └─ test_basic.py
    
```

Package directory

Python module directory

Marks this as a Python package

Example ROS2 node

Package metadata and dependencies

Python build and install instructions

Additional build config

Required file for ament package indexing

Launch file to start nodes

Parameter configuration file

Unit or integration test

```

ros2_ws/
└─ src/
    └─ my_cpp_pkg/
        └─ src/
            └─ my_node.cpp
        └─ include/
            └─ my_cpp_pkg/
                └─ my_node.hpp
        └─ package.xml
        └─ CMakeLists.txt
        └─ launch/
            └─ my_launch.py
        └─ config/
            └─ params.yaml
    
```

Package directory

C++ source code

Header file for public interfaces

Package metadata and dependencies

CMake build configuration

Launch file to start nodes

Parameter configuration file

How to make a ROS2 Package



4 ROS2 TUTORIAL

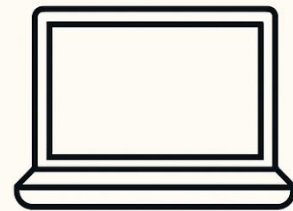
**Create a ROS2
Python Package**



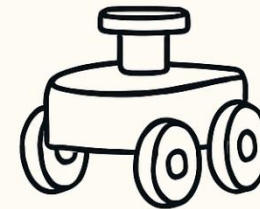
Class Activity: Project



- **Activity Objective:**
 - **Create a new ROS2 workspace, package, add a simple publisher node, build it, and run it from the workspace.**



Laptop



Mobile Robo

Project: Creating a workspace



- **Open a new terminal (terminal 1) and create our workspace, execute the following commands:**

```
mkdir -p ~/dev_ws/src
```

```
cd ~/dev_ws
```

```
colcon build --symlink-install
```

```
malubaidy@LAPTOP x malubaidy@LAPTOP x + v
malubaidy@LAPTOP-T13H099N:~$ mkdir -p ~/dev_ws/src
malubaidy@LAPTOP-T13H099N:~$ cd ~/dev_ws
malubaidy@LAPTOP-T13H099N:~/dev_ws$ colcon build --symlink-install

Summary: 0 packages finished [0.14s]
malubaidy@LAPTOP-T13H099N:~/dev_ws$
```

- **Open a new terminal (terminal 2) and create our workspace, execute the following commands:**

```
mkdir -p ~/robot_ws/src
```

```
cd ~/robot_ws
```

```
colcon build --symlink-install
```

```
malubaidy@LAPTOP x malubaidy@LAPTOP x + v
malubaidy@LAPTOP-T13H099N:~$ mkdir -p ~/robot_ws/src
malubaidy@LAPTOP-T13H099N:~$ cd ~/robot_ws
malubaidy@LAPTOP-T13H099N:~/robot_ws$ colcon build --symlink-install

Summary: 0 packages finished [0.08s]
malubaidy@LAPTOP-T13H099N:~/robot_ws$
```

Terminal 1

Project: Creating a Package



- Create an empty package (terminal 1 only)

```
Cd ..
```

```
source /opt/ros/jazzy/setup.bash
```

```
cd ~/dev_ws/src
```

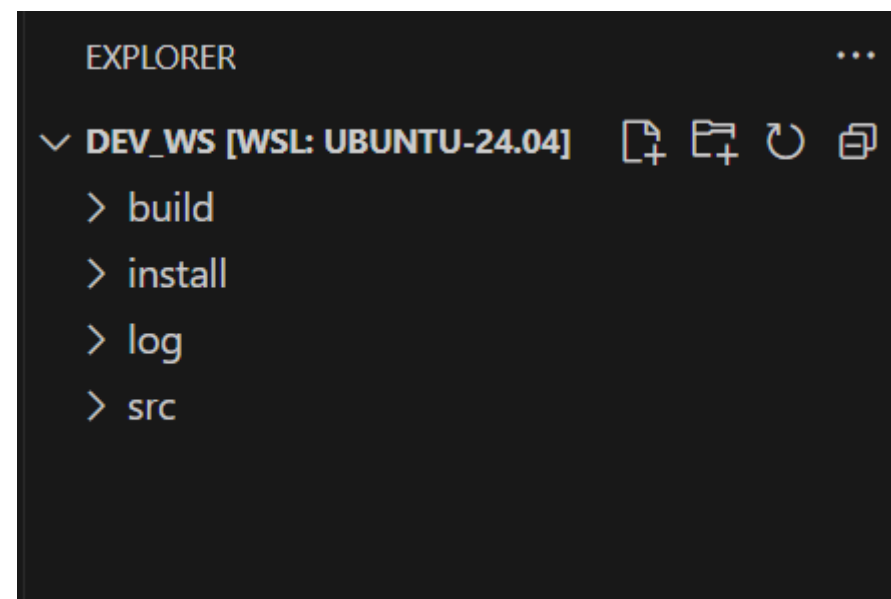
```
ros2 pkg create --build-type ament_cmake my_package
```

```
Cd ..
```

- Launch VS by typing:

```
Code .
```

```
malubaidy@LAPTOP-T13H099N:~/dev_ws/src$ code .
```

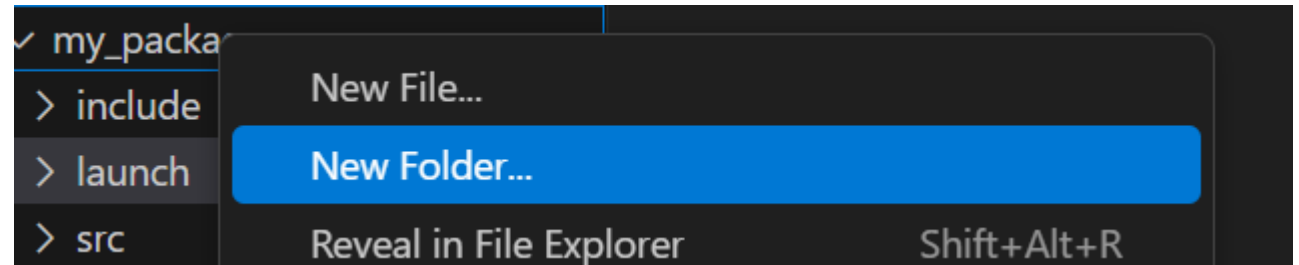


Project: Add our files



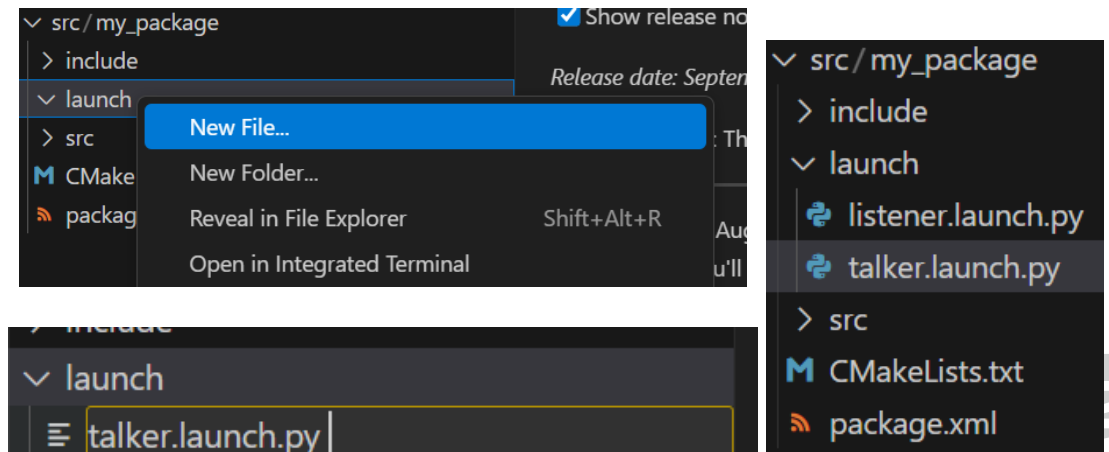
- **Create a directory called launch:**

- Right click on my_package
- Click “new folder”
- Name it: launch



- **Inside launch create two files, talker.launch.py and listener.launch.py.**

- Right click on launch folder and click on New File
- Type: talker.launch.py
- Repeat for: listener.launch.py



Project: Add our files



- **Now we can fill the files with the following content:**
 - **talker.launch.py**

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='demo_nodes_cpp',
            executable='talker'
        )
    ])
```

Project: Add our files



- **Now we can fill the files with the following content:**
 - **listener.launch.py**

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='demo_nodes_py',
            executable='listener'
        )
    ])
```

Project: Update the CMakeLists



- Add the following lines near the end of CMakeLists.txt, just before ament_package():

```
install(DIRECTORY launch
        DESTINATION share/${PROJECT_NAME}
)
```

```
src > my_package > CMakeLists.txt
9   find_package(ament_cmake REQUIRED)
10  # uncomment the following section in order to fill in
11  # further dependencies manually.
12  # find_package(<dependency> REQUIRED)
13
14  if(BUILD_TESTING)
15      find_package(ament_lint_auto REQUIRED)
16      # the following line skips the linter which checks for copyright
17      # comment the line when a copyright and license is added to all
18      set(ament_cmake_copyright_FOUND TRUE)
19      # the following line skips cpplint (only works in a git repo)
20      # comment the line when this package is in a git repo and when
21      # a copyright and license is added to all source files
22      set(ament_cmake_cpplint_FOUND TRUE)
23      ament_lint_auto_find_test_dependencies()
24  endif()
25
26  install(DIRECTORY launch
27          DESTINATION share/${PROJECT_NAME}
28  )
29
30  ament_package()
31
```

Project: Update other package info



- Add the following lines to package.xml, before the export section.

```
<exec_depend>demo_nodes_cpp</exec_depend>  
<exec_depend>demo_nodes_py</exec_depend>
```

```
1 <?xml version="1.0"?>  
2 <?xml-model href="http://download.ros.org/schema/package_format3."  
3 <package format="3">  
4   <name>my_package</name>  
5   <version>0.0.0</version>  
6   <description>TODO: Package description</description>  
7   <maintainer email="malubaidy@todo.todo">malubaidy</maintainer>  
8   <license>TODO: License declaration</license>  
9  
10  <buildtool_depend>ament_cmake</buildtool_depend>  
11  
12  <test_depend>ament_lint_auto</test_depend>  
13  <test_depend>ament_lint_common</test_depend>  
14  
15  <exec_depend>demo_nodes_cpp</exec_depend>  
16  <exec_depend>demo_nodes_py</exec_depend>  
17  
18  <export>  
19    <build_type>ament_cmake</build_type>  
20  </export>  
21 </package>  
22
```

Project: Build the package



- change directory back to our workspace

```
cd ~/dev_ws
```

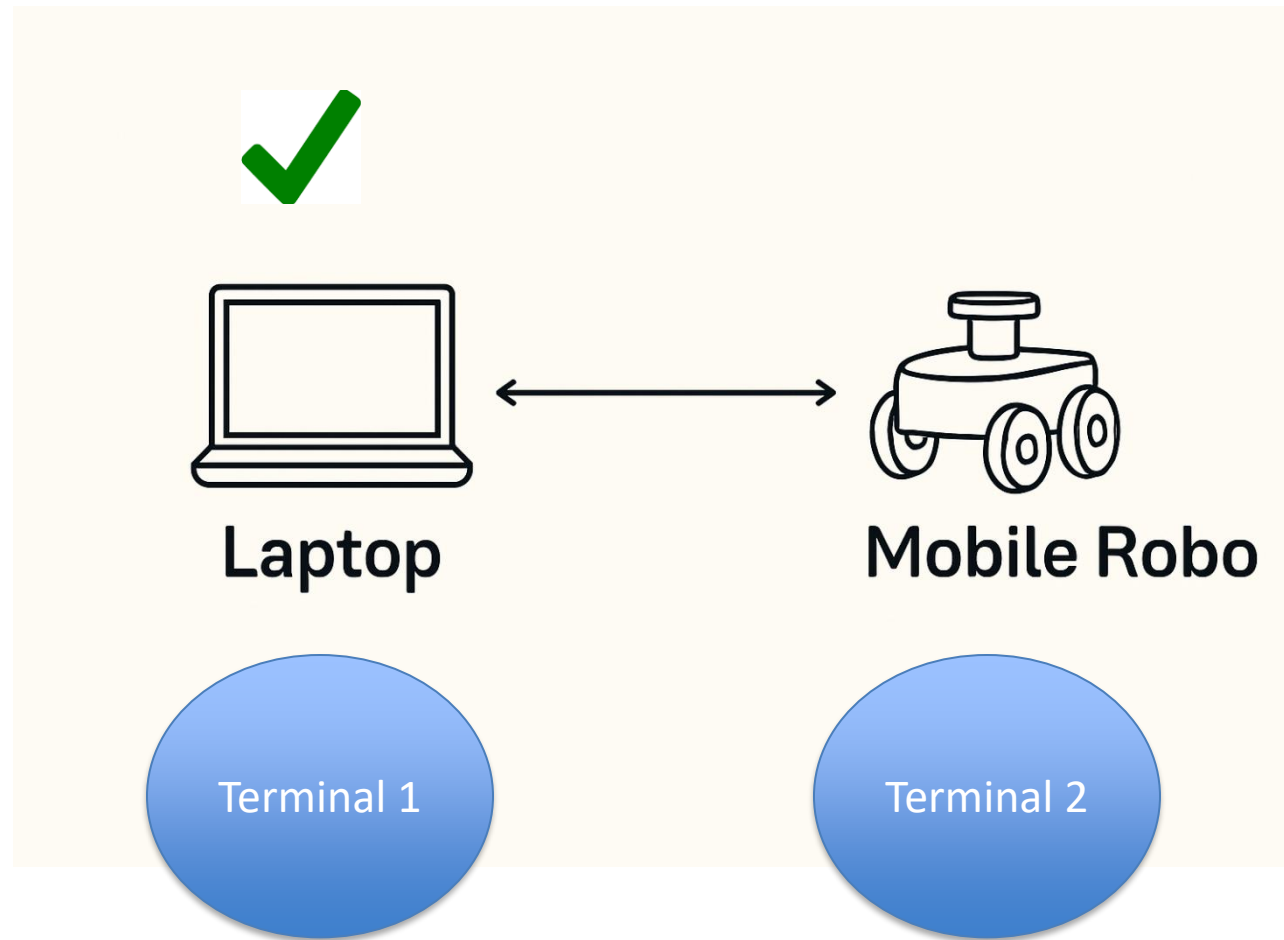
- Run:

```
colcon build --symlink-install
```

```
malubaidy@LAPTOP-T13H099N:~/dev_ws$ cd ~/dev_ws
malubaidy@LAPTOP-T13H099N:~/dev_ws$ colcon build --symlink-install
Starting >>> my_package
Finished <<< my_package [1.41s]

Summary: 1 package finished [1.66s]
malubaidy@LAPTOP-T13H099N:~/dev_ws$
```

Project: Next steps



Terminal 2

Project: Creating a Package



- Create an empty package (terminal 2only)

```
Cd ..
```

```
source /opt/ros/jazzy/setup.bash
```

```
cd ~/robot_ws/src
```

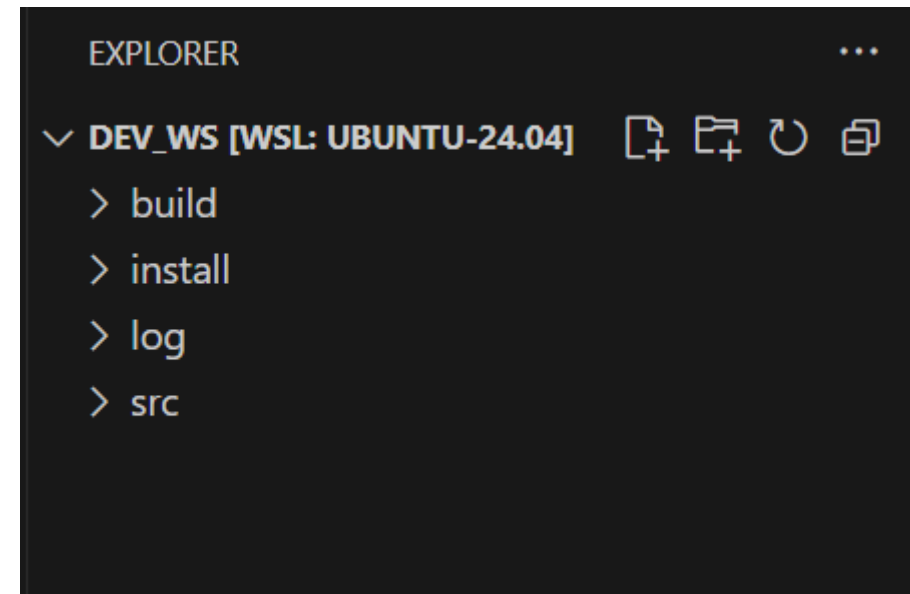
```
ros2 pkg create --build-type ament_cmake my_package
```

```
Cd ..
```

- Launch VS by typing:

```
Code .
```

```
malubaidy@LAPTOP-T13H099N:~/dev_ws/src$ code .
```

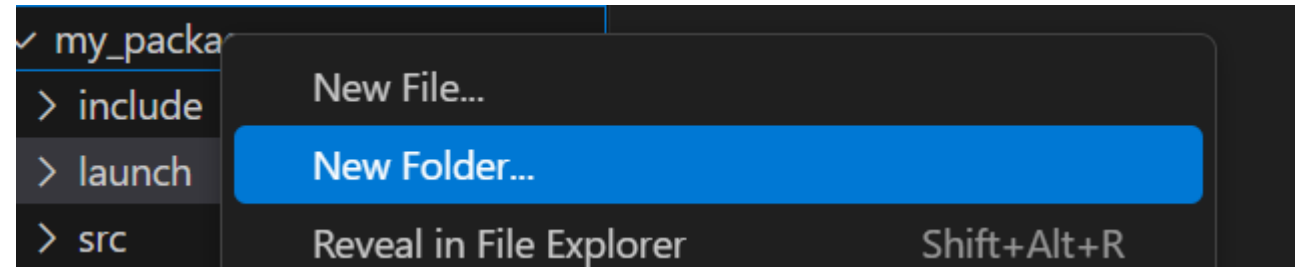


Project: Add our files



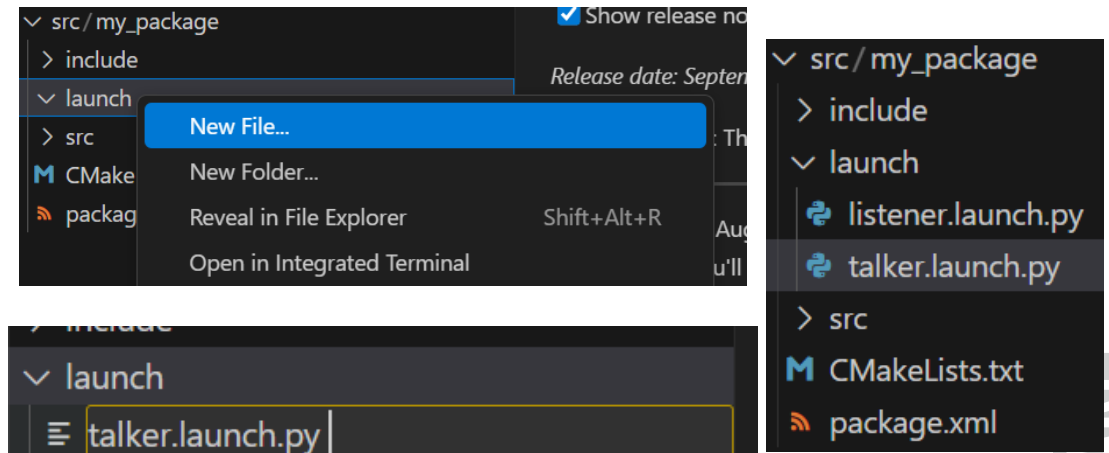
- **Create a directory called launch:**

- Right click on my_package
- Click “new folder”
- Name it: launch



- **Inside launch create two files, talker.launch.py and listener.launch.py.**

- Right click on launch folder and click on New File
- Type: talker.launch.py
- Repeat for: listener.launch.py



Project: Add our files



- **Now we can fill the files with the following content:**
 - **talker.launch.py**

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='demo_nodes_cpp',
            executable='talker'
        )
    ])
```

Project: Add our files



- **Now we can fill the files with the following content:**
 - **listener.launch.py**

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='demo_nodes_py',
            executable='listener'
        )
    ])
```

Project: Update the CMakeLists



- Add the following lines near the end of CMakeLists.txt, just before ament_package():

```
install(DIRECTORY launch
        DESTINATION share/${PROJECT_NAME}
)
```

```
src > my_package > CMakeLists.txt
9   find_package(ament_cmake REQUIRED)
10  # uncomment the following section in order to fill in
11  # further dependencies manually.
12  # find_package(<dependency> REQUIRED)
13
14  if(BUILD_TESTING)
15      find_package(ament_lint_auto REQUIRED)
16      # the following line skips the linter which checks for copyright
17      # comment the line when a copyright and license is added to all
18      set(ament_cmake_copyright_FOUND TRUE)
19      # the following line skips cpplint (only works in a git repo)
20      # comment the line when this package is in a git repo and when
21      # a copyright and license is added to all source files
22      set(ament_cmake_cpplint_FOUND TRUE)
23      ament_lint_auto_find_test_dependencies()
24  endif()
25
26  install(DIRECTORY launch
27          DESTINATION share/${PROJECT_NAME}
28  )
29
30  ament_package()
31
```

Project: Update other package info



- Add the following lines to package.xml, before the export section.

```
<exec_depend>demo_nodes_cpp</exec_depend>  
<exec_depend>demo_nodes_py</exec_depend>
```

```
1 <?xml version="1.0"?>  
2 <?xml-model href="http://download.ros.org/schema/package_format3."  
3 <package format="3">  
4   <name>my_package</name>  
5   <version>0.0.0</version>  
6   <description>TODO: Package description</description>  
7   <maintainer email="malubaidy@todo.todo">malubaidy</maintainer>  
8   <license>TODO: License declaration</license>  
9  
10  <buildtool_depend>ament_cmake</buildtool_depend>  
11  
12  <test_depend>ament_lint_auto</test_depend>  
13  <test_depend>ament_lint_common</test_depend>  
14  
15  <exec_depend>demo_nodes_cpp</exec_depend>  
16  <exec_depend>demo_nodes_py</exec_depend>  
17  
18  <export>  
19    <build_type>ament_cmake</build_type>  
20  </export>  
21 </package>  
22
```

Project: Build the package



- **change directory back to our workspace**

```
cd ~/robot_ws
```

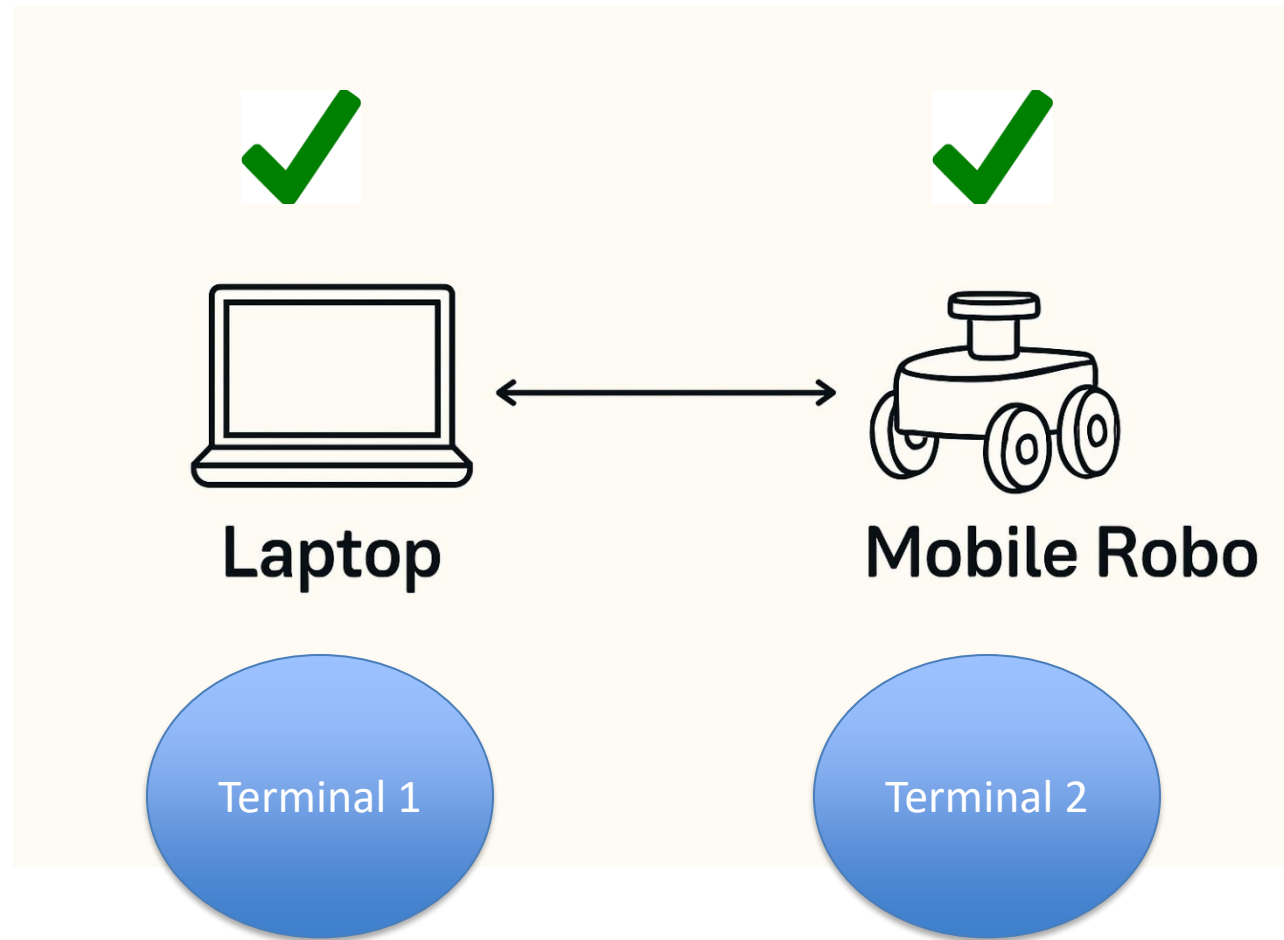
- **Run:**

```
colcon build --symlink-install
```

```
malubaidy@LAPTOP-T13H099N:~/dev_ws$ cd ~/dev_ws
malubaidy@LAPTOP-T13H099N:~/dev_ws$ colcon build --symlink-install
Starting >>> my_package
Finished <<< my_package [1.41s]

Summary: 1 package finished [1.66s]
malubaidy@LAPTOP-T13H099N:~/dev_ws$
```

Project: Next steps



Project: Executing



- **Terminal 1 dev side:**

```
cd ~
```

```
source /opt/ros/jazzy/setup.bash
```

```
cd dev_ws
```

```
source install/setup.bash
```

```
ros2 launch my_package talker.launch.py
```

- **Terminal 2 robot side:**

```
cd ~
```

```
source /opt/ros/jazzy/setup.bash
```

```
cd robot_ws
```

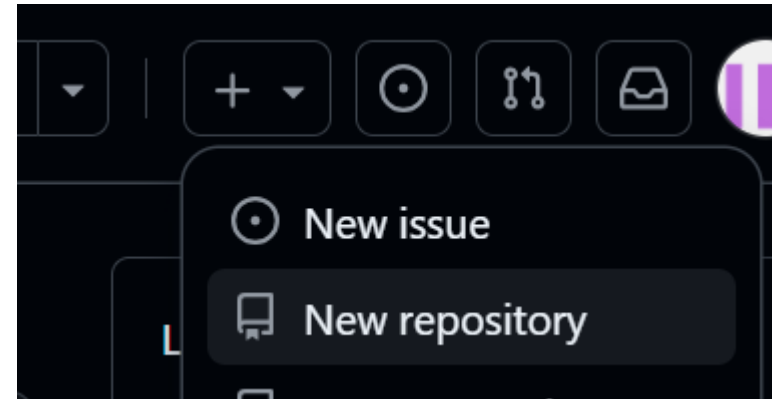
```
source install/setup.bash
```

```
ros2 launch my_package listener.launch.py
```

Project: Creating a repo



- Go to <https://github.com/>
- Create an account
- Login
- Click the + sign in the upper right corner



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

General

Owner * malubaidy / Repository name * my_package
✔ my_package is available.

Great repository names are short and memorable. How about **stunning-octo-winner**?

Description

class activity
14 / 350 characters

Configuration

Choose visibility *
Choose who can see and commit to this repository Public

Add README

What is a Launch File



ROS2 PYTHON
LAUNCH FILE



Debugging and Common Errors



Issue	Cause	Fix
command not found	Environment not sourced	source install/setup.bash
ImportError	Missing dependency	Add to package.xml
No executable	Not marked chmod +x	chmod +x my_script.py
Build failed	Syntax or missing file	Check logs in log/ directory

Key things to remember...



- ***Key things to remember***
 - i. *What is a ROS2 workspace and why it is important.*
 - ii. *What is a ROS2 package and how to setup a package.*
- ***Practicing outside the classroom***

Read the following articles to better understand workspaces

- <https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html>
- <https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html>

Summary



- **This week we covered Performance objectives**
 - Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions
 - Configure ROS2 on a Linux-based system
 - Develop ROS2 nodes in Python or C++ using rclpy or rclcpp
 - Manage ROS2 workspaces, packages and files by applying structured software development and modular design practices
- **Next week's topics**
 - What is a ROS2 workspace?
 - What is a ROS2 package?
- **Next week's performance objectives**
 - Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions
 - Configure ROS2 on a Linux-based system
 - Develop ROS2 nodes in Python or C++ using rclpy or rclcpp
 - Manage ROS2 workspaces, packages and files by applying structured software development and modular design practices
 - Utilize ROS2 topics, services and actions for inter-node communication

THANK YOU.



WE ARE

HUMBER